Task 10 – Compare Expectations with Execution

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K4 – Analyse

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task10)

1. What I expected

Before running: (1) float and Decimal totals would agree to the paisa for 300 payments and differ by a few paise at 6,000 and 150,000 because float errors accumulate; (2) Decimal would be slower; (3) rounding each payment or rounding only the total would give about the same answer.

2. Experiment

compare.py generates n = 300, 6,000 and 150,000 random entries with a fixed seed (litres with two decimals, fat 2.5–6.5, SNF 7.8–9.2) and computes the ten-day total in three ways: **float** (milkrate.py from Task 7, each payment rounded with round()), **Decimal, each payment rounded to the paisa** (milkcalc.py from Task 9) and **Decimal, only the total rounded**. Times use time.perf_counter().

compare.py (excerpt)

```
import random, time
from decimal import Decimal as D
import milkrate as fl        # float version (Task 7)
import milkcalc as dc        # Decimal version (Task 9)

def make_entries(n, seed=2025):
    rnd = random.Random(seed)
    out = []
    for _ in range(n):
        litres = f"{rnd.uniform(2, 25):.2f}"
        fat = f"{rnd.randint(25, 65) / 10:.1f}"
        snf = f"{rnd.randint(78, 92) / 10:.1f}"
        out.append((litres, fat, snf))
    return out

def float_total(entries):
    total = 0.0
    for l, f, s in entries:
        total += fl.payout(float(l), float(f), float(s))
    return total

def decimal_total(entries):
    return dc.ten_day_total([(D(l), D(f), D(s)) for l, f, s in entries])

def decimal_round_only_total(entries):
    t = D(0)
    for l, f, s in entries:
        t += D(l) * dc.rate_per_litre(D(f), D(s))     # no per-payment rounding
    return dc.to_money(t)
```

Results

```
     n          float total    Decimal total     difference  round-total only gap (paise)
   300    143587.8100000001        143587.96  -0.1499999999         143587.87           -9
       float time     0.44 ms   Decimal time     1.10 ms
  6000   2772380.9299999960       2772383.76  -2.8300000040        2772380.75         -301
       float time     8.74 ms   Decimal time    20.52 ms
150000  69002202.1399991661      69002275.38 -73.2400008300       69002203.24        -7214
       float time   267.30 ms   Decimal time   524.34 ms
```

n	float total	Decimal total (bank)	float − Decimal	round only total	float time	Decimal time
300	143,587.81	143,587.96	−0.15	143,587.87 (−0.09)	0.44 ms	1.10 ms
6,000	2,772,380.93	2,772,383.76	−2.83	2,772,380.75 (−3.01)	8.7 ms	20.5 ms
150,000	69,002,202.14	69,002,275.38	−73.24	69,002,203.24 (−72.14)	267 ms	524 ms

3. Reasons for the difference – a second experiment

The gaps are much bigger than float accumulation error alone would give, so I separated the causes:

Same 150,000 entries with litres to 1 decimal and to 2 decimals

```
litres with 1 decimal(s): float total 69002544.220001  Decimal total 69002544.22  gap 0.0000  entries nee
litres with 2 decimal(s): float total 69002202.139999  Decimal total 69002275.38  gap -73.2400  entries n
```

- **Accumulation error is tiny:** with litres to one decimal every amount is already an exact number of paise; float and Decimal totals agree to the paisa (gap 0.0000, only 0.000001 of float noise on 69 million rupees).
- **The real cause is rounding of half-paise amounts.** With two-decimal litres an amount can fall exactly half-way between two paise, e.g. 0.15 L × Rs 30.10 = 4.515. Decimal rounds the tie up (Rs 4.52), as a bank does; with float, round(4.515, 2) gives 4.51 because the binary value is stored just below the tie, so one paisa less is paid. 7,324 of 150,000 payments differ by exactly one paisa: 7,324 × 0.01 = Rs 73.24, which is exactly the gap in the table.
- **Rounding only the total is wrong for the bank:** a bank statement is a sum of whole-paise credits. Not rounding each payment lets half-paise remainders vanish (or accumulate) and the total is Rs 72.14 away at 150,000 – a result no farmer was actually paid. **Rounding each payment, with an explicit half-up rule on exact decimals, gives the correct bank amount.**
- **Time:** both grow in proportion to n (300 → 6,000 is 20× and the time grows about 20×; 6,000 → 150,000 is 25× and time about 30×). Decimal is about twice as slow (≈ 3.5 µs against 1.8 µs per payment) but 0.5 s for 150,000 payments is acceptable for a job run once every ten days.

My first expectation was therefore partly wrong: float and Decimal already disagree at 300 payments, and for a reason (tie rounding) that is different from the one I predicted (accumulated error).

Alternative Concept and Complexity Analysis

Alternative: compute the rate for each distinct (fat, snf) pair once, keep it in a dictionary `RATE_TABLE[(fat, snf)]` and look it up for every entry.

ratetable.py (excerpt)

```
N = 150000
fats = [round(rnd.randint(25, 65) / 10, 1) for _ in range(N)]
snfs = [round(rnd.randint(78, 92) / 10, 1) for _ in range(N)]
pairs = list(zip(fats, snfs))

t0 = time.perf_counter()
for f, s in pairs:
    fl.rate_per_litre(f, s)
t1 = time.perf_counter()
table = {}
for f in range(25, 66):
    for s in range(78, 93):
        table[(f / 10, s / 10)] = fl.rate_per_litre(f / 10, s / 10)
t2 = time.perf_counter()
for p in pairs:
    table[p]
t3 = time.perf_counter()
print(f"float recalculation : {1e9*(t1-t0)/N:7.0f} ns/entry")
print(f"build table ({len(table)} pairs) : {1e6*(t2-t1):7.0f} us once")
print(f"float table lookup  : {1e9*(t3-t2)/N:7.0f} ns/entry")
```

Measured (ratetable.py)

```
float recalculation :     886 ns/entry
build table (615 pairs) :    2034 us once
float table lookup  :     187 ns/entry
Decimal recalculation:    2408 ns/entry
Decimal table lookup :     184 ns/entry  (table of 615 pairs built once)
dict size (bytes) for 615 pairs: 18512
```

	Recalculate for every entry	Rate table (dictionary)
Time per entry	float ≈ 886 ns; Decimal ≈ 2,408 ns – O(1) but with many operations	≈ 185 ns for both – O(1) hash lookup (4.7× faster for float, 13× for Decimal)
Extra space	O(1)	O(k) for k distinct pairs: 615 pairs ≈ 18.5 KB (at most 51 × 41 = 2,091 pairs for the whole allowed range)
One-time cost	none	building 615 float pairs ≈ 2 ms
Break-even	—	2,034 µs / (886 − 187) ns ≈ 2,900 entries; a single centre has about 300 × 20 = 6,000 entries per cycle, so it already pays off

For n entries: recalculation costs O(n) with a large constant; the table costs O(k + n) with a tiny constant, where k ≤ 2,091 is independent of n. Because k is fixed and small while n grows to millions, the table is the better design for the district union – provided it is rebuilt whenever the rate card changes (a stale table would reintroduce the “partly applied revision” bug).

Evidence

Measured totals and times above for 300, 6,000 and 150,000 payments, with the reason for every difference. Source: Task10/compare.py, ratetable.py.